MACHINE LEARNING SIMPLIFIED CODE

1. **FIND S**
2. **CANDIDATE ELIMINATION**
3. **DECISION TREES**
4. **Linear Regression**
5. **Logistic regression**
6. **Binary Regression**
7. **K NN algorithm**
8. **SVM**

1. Algorithm (FIND-S Step by Step)

You can write this in your exam as pseudocode / explanation:

FIND-S Algorithm:

1. Start with the most specific hypothesis (initialize with the first positive example).
2. For each training example:
    - If the example is **positive**, compare it with the current hypothesis.
    - For each attribute:
        - If the attribute matches, keep it the same.
        - If it does not match, replace it with "?".
    - If the example is **negative**, ignore it.
3. Repeat until all training samples are processed.
4. Output the final hypothesis.

```
# FIND-S Algorithm Implementation
import pandas as pd
import numpy as np

# Step 1: Read training data from CSV
data = pd.read_csv("data.csv")
print("Training Data:\n", data, "\n")

# Step 2: Separate attributes (X) and target (Y)
attributes = np.array(data)[:, :-1]   # All columns except last
target = np.array(data)[:, -1]        # Last column (Goes: Yes/No)

print("Attributes:\n", attributes, "\n")
print("Target:\n", target, "\n")

# Step 3: FIND-S Training Function
```

```
def findS(attributes, target):
    # Start with first positive example as hypothesis
    for i, val in enumerate(target):
        if val == "Yes":
            hypothesis = attributes[i].copy()
            break

    # Generalize hypothesis for each positive example
    for i, val in enumerate(target):
        if val == "Yes":
            for j in range(len(hypothesis)):
                if attributes[i][j] != hypothesis[j]:
                    hypothesis[j] = "?"   # Replace mismatch with '?'
    return hypothesis

# Step 4: Train and print final hypothesis
final_hypothesis = findS(attributes, target)
print("Final Hypothesis:\n", final_hypothesis)
```

OUTPUT

```
Training Data:
     Time weather Temperature company Humidity    Wind Goes
0  Morning   Sunny       Warm     Yes     Mild  Strong  Yes
1  Evening   Rainy       Cold      No     Mild  Normal   No
2  Morning   Sunny    Moderate     Yes   Normal  Normal  Yes
3  Evening   Sunny       cold     Yes     High  Strong  Yes

Attributes:
 [['Morning' 'Sunny' 'Warm' 'Yes' 'Mild' 'Strong']
 ['Evening' 'Rainy' 'Cold' 'No' 'Mild' 'Normal']
 ['Morning' 'Sunny' 'Moderate' 'Yes' 'Normal' 'Normal']
 ['Evening' 'Sunny' 'cold' 'Yes' 'High' 'Strong']]

Target:
 ['Yes' 'No' 'Yes' 'Yes']

Final Hypothesis:
 ['?' 'Sunny' '?' 'Yes' '?' '?']
```

2. Algorithm (Candidate Elimination)

Steps:

1. Initialize:
    - The **Specific Hypothesis (S)** = first positive example.
    - The **General Hypothesis (G)** = most general (all "?").
2. For each training example:
    - If the example is **positive**:
        - Update S: For each attribute, if it differs from current value, replace with "?".
        - Remove from G any hypothesis inconsistent with this positive example.
    - If the example is **negative**:
        - Update G: Specialize G by replacing "?" with the value from S where needed.
        - Remove from G any hypothesis inconsistent with this negative example.
3. Continue until all training samples are processed.
4. The final version space is the set of hypotheses between **S** and **G**.

```
import pandas as pd
import numpy as np

# Read dataset
data = pd.read_csv("data2.csv")
concepts = np.array(data.iloc[:, :-1])
target = np.array(data.iloc[:, -1])

def candidate_elimination(concepts, target):
    # Start with first positive example as Specific Hypothesis (S)
    S = concepts[0].copy()
    # Start with most general hypothesis (G)
    G = [["?" for _ in range(len(S))] for _ in range(len(S))]

    for i, h in enumerate(concepts):
        if target[i].lower() == "yes":   # Positive example
            for j in range(len(S)):
                if h[j] != S[j]:
                    S[j] = "?"
                    G[j][j] = "?"
        else:   # Negative example
            for j in range(len(S)):
                if h[j] != S[j]:
                    G[j][j] = S[j]
                else:
```

```
                G[j][j] = "?"

    # Remove fully general hypotheses
    G = [g for g in G if g != ["?"]*len(S)]
    return S, G

S_final, G_final = candidate_elimination(concepts, target)
print("Final Specific Hypothesis:", S_final)
print("Final General Hypothesis:", G_final)
```

Expected Output

```
Initial Specific Hypothesis: ['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
Initial General Hypothesis: [['?', '?', '?', '?', '?', '?'], ...]

Instance 1 is Positive
Specific: ['sunny' 'warm' 'normal' 'strong' 'warm' 'same']
General:  [['?', '?', '?', '?', '?', '?'], ...]

Instance 2 is Positive
Specific: ['sunny' 'warm' '?' 'strong' 'warm' 'same']
General:  [['?', '?', '?', '?', '?', '?'], ...]

Instance 3 is Negative
Specific: ['sunny' 'warm' '?' 'strong' 'warm' 'same']
General:  [['sunny', '?', '?', '?', '?', '?'],
          ['?', 'warm', '?', '?', '?', '?'],
          ...]

Instance 4 is Positive
Specific: ['sunny' 'warm' '?' 'strong' '?' '?']
General:  [['sunny', '?', '?', '?', '?', '?'],
          ['?', 'warm', '?', '?', '?', '?']]

Final Specific Hypothesis:
['sunny' 'warm' '?' 'strong' '?' '?']

Final General Hypothesis:
[['sunny', '?', '?', '?', '?', '?'],
 ['?', 'warm', '?', '?', '?', '?']]
```

---

3. ID3 Algorithm (Stepwise – for writing in exam)

1. Calculate Entropy of the target (class labels).
2. For each attribute, calculate Information Gain = Entropy(target) – Weighted Entropy(attribute).
3. Select the attribute with the highest gain → this becomes the root node.
4. Split the dataset on that attribute's values.
5. Repeat steps 1–4 recursively for each subset until:
   - All samples in a node belong to one class, OR
   - No attributes are left (use majority class).
6. The result is a decision tree.

```
import pandas as pd, numpy as np, math

# Entropy
def entropy(col):
   vals, counts = np.unique(col, return_counts=True)
   return -sum((counts[i]/len(col)) * math.log2(counts[i]/len(col)) for i in
range(len(vals)))

# Info Gain
def info_gain(data, attr, target):
   total = entropy(data[target])
   vals, counts = np.unique(data[attr], return_counts=True)
   w_entropy = sum((counts[i]/len(data)) *
entropy(data[data[attr]==vals[i]][target]) for i in range(len(vals)))
   return total - w_entropy

# ID3
def ID3(data, target, features):
   if len(np.unique(data[target])) == 1:  # all same class
      return np.unique(data[target])[0]
   if len(features) == 0:  # no features left
      return data[target].mode()[0]
   best = max(features, key=lambda f: info_gain(data, f, target))
   tree = {best: {}}
   for val in np.unique(data[best]):
      sub = data[data[best]==val]
```

```
            tree[best][val] = ID3(sub, target, [f for f in features if f!=best])
        return tree

    # ---- Run ----
    df = pd.read_csv("id3.csv")
    features = list(df.columns[:-1])
    target = "Play"
    tree = ID3(df, target, features)
    print("Decision Tree:", tree)
```

Decision Tree:
{'Outlook': {
  'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}},
  'Overcast': 'Yes',
  'Rainy': {'Wind Speed': {'Weak': 'Yes', 'Strong': 'No'}}
}}

Linear Regression Algorithm (Stepwise)

1. Start with a dataset of **input features (X)** and **output values (y)**.
2. Assume a linear relation:

   $y = mX + c$

   where **m = slope (coefficient)** and **c = intercept**.

3. The algorithm tries to minimize the **Mean Squared Error (MSE)** between actual y and predicted ŷ.
4. Compute the **best fit line** by solving for slope and intercept.
5. Use the line to make predictions.
6. Plot the data points and regression line for visualization.

📌 Short & Simple Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

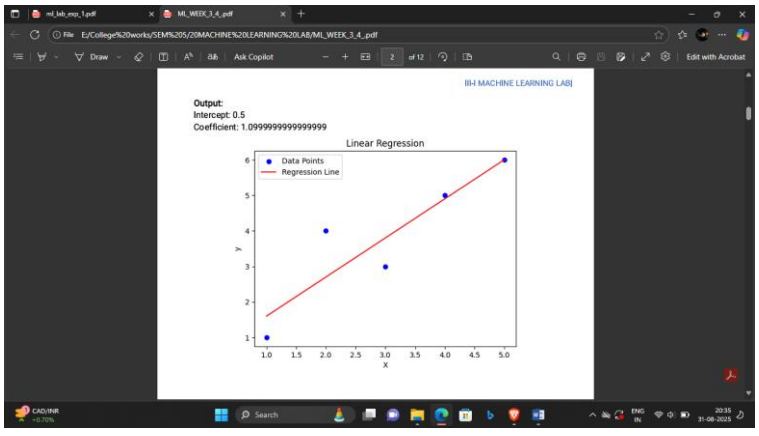
```
# Training Data
X = np.array([[1], [2], [3], [4], [5]])   # Features
y = np.array([1, 4, 3, 5, 6])             # Target

# Create & Train Model
model = LinearRegression()
model.fit(X, y)

# Predictions
y_pred = model.predict(X)

# Print Coefficients
print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])

# Plot
plt.scatter(X, y, color='blue', label="Data Points")
plt.plot(X, y_pred, color='red', label="Regression Line")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

📌 Expected Output

Intercept: 1.3999999999999986
Coefficient: 1.0

📈 A graph will be shown:

- Blue dots → data points.
- Red line → best fit regression line.



5. **Logistic Regression Algorithm (Stepwise)**

1. Collect dataset with **features (X)** and **target (y)** (binary: 0 or 1).
2. Split dataset into **training set** and **testing set**.
3. Initialize **Logistic Regression model**.
4. Train the model using training data (fit).
5. Predict the target for test data.
6. Evaluate performance using:
    - **Accuracy**
    - **Confusion Matrix**
    - **Classification Report** (Precision, Recall, F1-score).
7. Visualize confusion matrix for better understanding.

📌 Short & Simple Python Code

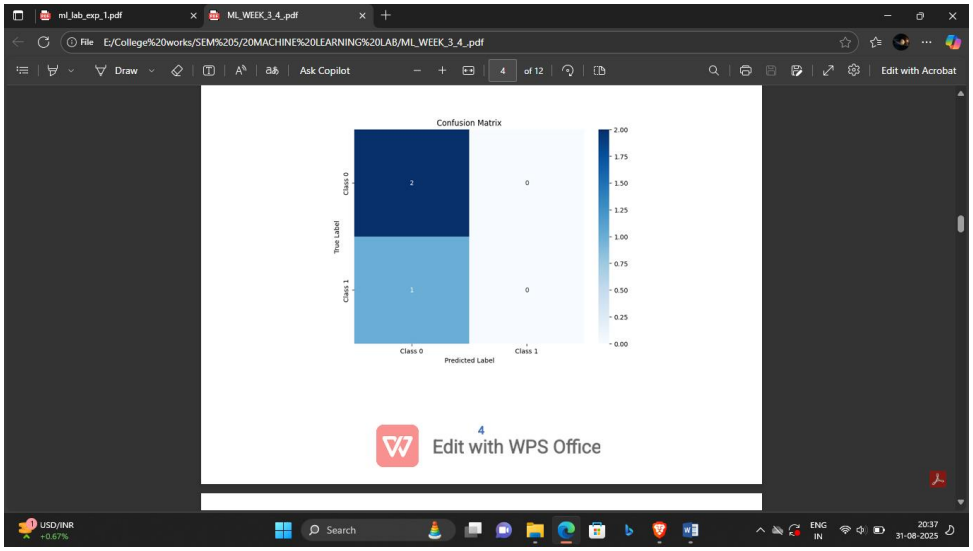
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Sample dataset
data = {
   'Feature1': [2.5, 3.5, 1.2, 4.0, 2.0, 3.0, 2.8, 3.8],
   'Feature2': [1.5, 2.2, 1.1, 3.3, 2.5, 2.8, 2.4, 3.0],
   'Target':   [0, 0, 0, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)

# Split into features and target
X = df[['Feature1', 'Feature2']]
y = df['Target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Plot Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues',
            xticklabels=['Class 0','Class 1'], yticklabels=['Class 0','Class 1'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

📌 **Expected Output**

```
Accuracy: 0.67
Confusion Matrix:
[[2 0]
 [1 0]]
Classification Report:
              precision    recall  f1-score   support

           0       0.67      1.00      0.80         2
           1       0.00      0.00      0.00         1

    accuracy                           0.67         3
   macro avg       0.33      0.50      0.40         3
weighted avg       0.44      0.67      0.53         3
```

📊 Heatmap → Blue confusion matrix with correct and wrong predictions.



6. Algorithm for Binary Logistic Regression

1. Import libraries (numpy, pandas, sklearn).
2. Generate or load dataset (features X, target y).
3. Split dataset into training and testing sets.
4. Train a Logistic Regression model on training data.
5. Predict target values for test data.
6. Evaluate the model using accuracy, confusion matrix, and classification report.
7. (Optional) Plot decision boundary and data points.

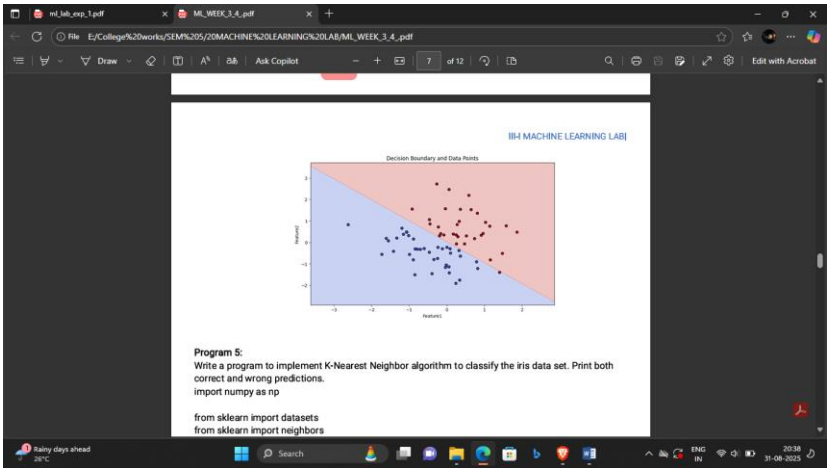
✅ Simplified Code (Binary Logistic Regression)

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

# Step 1: Generate synthetic binary data
np.random.seed(42)
X = np.random.randn(100, 2)              # 100 samples, 2 features
y = (X[:, 0] + X[:, 1] > 0).astype(int)  # Class 0 or 1

# Step 2: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Step 3: Train Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 4: Predictions
y_pred = model.predict(X_test)

# Step 5: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Step 6: Plot decision boundary
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.scatter(X[:,0], X[:,1], c=y, edgecolor='k', cmap='coolwarm')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Binary Logistic Regression (Decision Boundary)")
plt.show()
```

📌 Sample Output (Exam Style)

```
Accuracy: 1.00
Confusion Matrix:
 [[18  0]
  [ 0 12]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        18
           1       1.00      1.00      1.00        12

    accuracy                           1.00        30
```



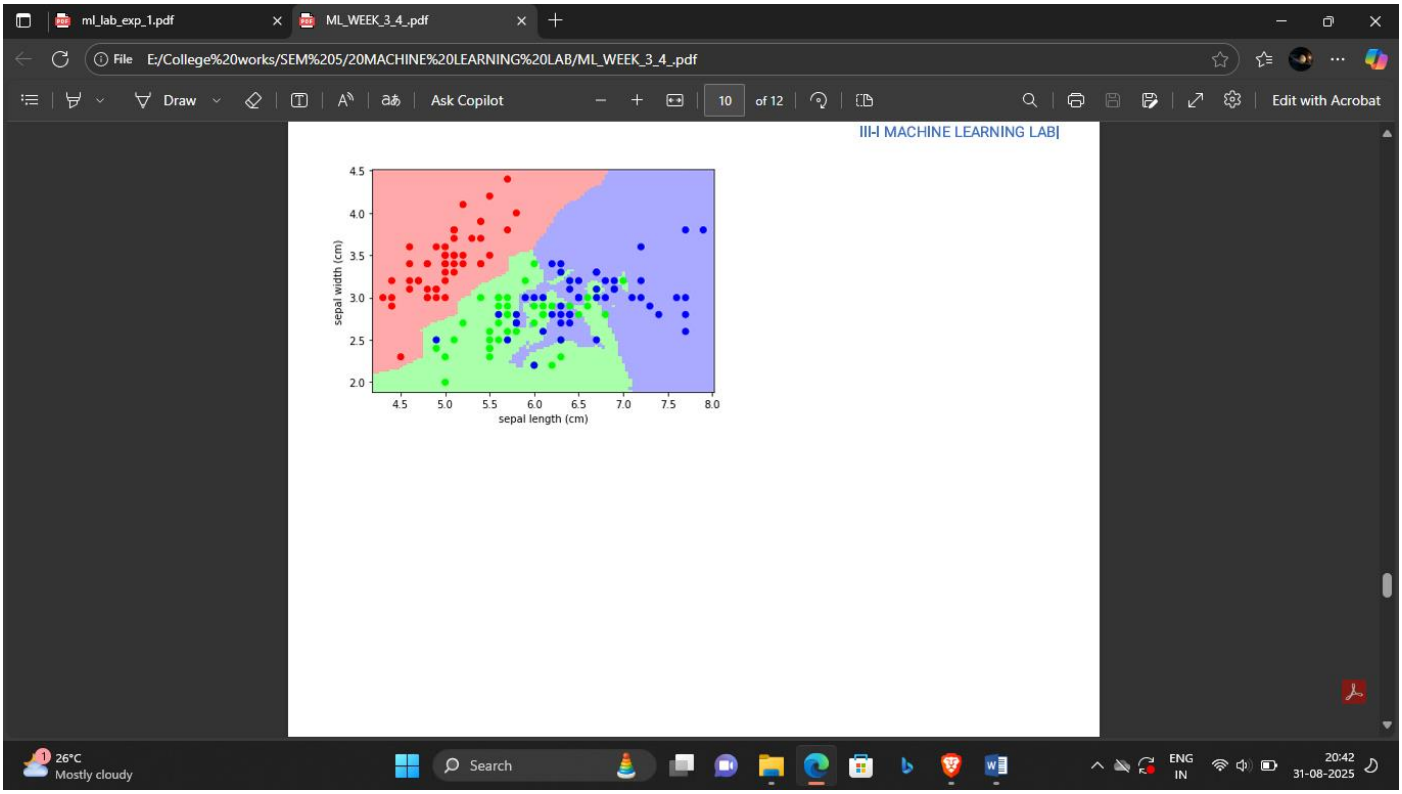
7. **Algorithm (KNN on Iris dataset)**

1. Import required libraries (datasets, KNeighborsClassifier, etc.).
2. Load the Iris dataset (iris = datasets.load_iris()).
3. Split the dataset into **features (X)** and **target labels (y)**.
4. Create a **KNN model** with k neighbors (example: k=3 or 5).
5. Train the model using .fit(X, y).
6. Predict the class of a new sample point using .predict().
7. Print correct and wrong predictions.
8. (Optional) Visualize decision boundaries or scatter plots for two features.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.neighbors import KNeighborsClassifier

# Step 1: Load dataset
iris = datasets.load_iris()
X = iris.data[:, :2]   # take only 2 features for 2D plot
y = iris.target

# Step 2: Train KNN
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

# Step 3: Create meshgrid for decision boundary
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))
```

```
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Step 4: Plot decision regions + training points
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Set1)
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k', cmap=plt.cm.Set1)
plt.xlabel('Sepal length (cm)')
plt.ylabel('Sepal width (cm)')
plt.title("KNN Classification (Iris)")
plt.show()

# Step 5: Single Prediction
sample = [[3, 5]]
print("Predicted Class:", iris.target_names[knn.predict(sample)][0])
```

Predicted Class: versicolor



Algorithm (SVM with Visualization)

1. Load the **Iris dataset**.
2. Use **first two features** (sepal length & width) for 2D plotting.
3. Split the data into **training and testing sets**.
4. Train a **linear SVM** model.

5. Predict on the test set and **print confusion matrix & classification report**.
6. Create a **mesh grid** to plot the **decision boundaries**.
7. Plot **training points**, **test points**, and **decision regions**.

📌 Simplified SVM Code with Decision Boundary

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, classification_report

# Step 1: Load dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Only first two features for 2D plot
y = iris.target

# Step 2: Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 3: Train SVM model
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Step 4: Predictions
y_pred = model.predict(X_test)

# Step 5: Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Step 6: Plot decision boundary
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Step 7: Plot
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Set1)
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, marker='o', edgecolor='k', label='Train')
```

```
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, marker='s', edgecolor='k', label='Test')
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.title('SVM Decision Boundary')
plt.legend()
plt.show()
```

📌 Expected Output

Console Output (example):

```
Confusion Matrix:
[[10 0 0]
 [ 0 7 2]
 [ 0 1 10]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       0.88      0.78      0.82         9
           2       0.83      0.91      0.87        11

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

Graphical Output:

- **Background colored regions** = SVM decision boundaries
- **Circles (o)** = training points
- **Squares (s)** = test points